Colaborando no GitHub

= como várias pessoas constroem soluções juntas

```
$ aula --data 23/10/2026 --instrutor "Guilherme Medina (Thompson Tecnologia)"
```

Hoje cada um de vocês colocou um **cartão** num site de verdade, no ar para qualquer pessoa ver: **medinagui.github.io/mural-ams-2026**. O mais importante não é o cartão: é **o caminho** que ele fez, o mesmo que equipes de tecnologia do mundo inteiro usam para trabalhar juntas:

01 Você faz uma **cópia** do projeto na sua conta (*fork*).

02 Você **cria um arquivo** com o seu cartão e salva (*commit*).

03 Você **pede para juntar** o seu arquivo ao projeto (*pull request*).

04 Um **robô** confere se está tudo certo e responde ✅ ou ❌.

05 O instrutor **revisa e aprova** (*merge*).

06 O site **se atualiza sozinho** e o seu cartão aparece no mural.

Mini dicionário

PALAVRA	O QUE É	COMPARANDO COM…
Git	Programa que guarda o histórico de uma pasta: cada versão, quem mudou, quando e por quê	= A câmera que fotografa cada versão
GitHub	Site onde as pessoas guardam e compartilham projetos que usam Git	= O Instagram onde as fotos ficam e todo mundo comenta
Repositório	A pasta do projeto no GitHub, com todo o histórico	= Uma pasta compartilhada que lembra de tudo
Commit	Salvar uma mudança, com uma mensagem dizendo o que mudou	= Salvar o jogo (checkpoint)
Histórico	A lista de todos os commits: quem, quando e o quê	= A lista de saves do jogo
Branch	Uma linha do tempo separada para testar sem estragar a principal (a `main`)	= Um universo paralelo
Fork	A sua cópia do projeto, na sua conta	= "Fazer uma cópia" de um Google Docs de outra pessoa
Pull request	Pedido para juntar a sua mudança ao projeto original	= Entregar o trabalho para o professor corrigir
Merge	Juntar de vez a mudança ao projeto	= O trabalho corrigido indo para o mural da sala
Conflito	Duas pessoas mudaram a mesma linha; um humano decide qual fica	= Dois alunos escrevendo no mesmo lugar do cartaz
O robô	Programa que roda sozinho a cada PR e confere as regras (GitHub Actions)	= O fiscal da prova que confere se você preencheu o nome
GitHub Pages	Serviço do GitHub que transforma arquivos em site	= A vitrine da loja

Antes de começar

CHECKLIST

- Conta no GitHub e você **logado**. Não tem? Crie em **github.com/signup** (é grátis).
- **E-mail confirmado**. Sem isso, o GitHub não deixa abrir pull request.
- Tudo é feito **pelo navegador**. Não precisa instalar nada.

Regras do cartão

CAMPO	O QUE COLOCAR	REGRAS
`nome`	Seu primeiro nome	Só o primeiro nome, **sem sobrenome**. Até 20 letras. Pode ter acento e hífen (`Ana-Clara`).
`emoji`	1 emoji que tenha a sua cara	Exatamente **1** emoji. No Windows: tecla **Windows + .** (ponto).
`frase`	Complete: *"Com tecnologia eu quero criar..."*	Até **80 caracteres**. Sem links, telefone ou e-mail. Sem palavrão.
`cor`	A cor do seu cartão	`vermelho` `laranja` `amarelo` `verde` `turquesa` `azul` `roxo` `rosa`

Modelo para copiar

```
{
  "nome": "Ana",
  "emoji": "🎮",
  "frase": "Com tecnologia eu quero criar
jogos que ensinam matemática",
  "cor": "rosa"
}
```

Troque só o que está **entre aspas do lado direito**. Mantenha as **aspas** `"` e as **vírgulas** no fim das linhas, **menos na última** antes do `}`.

Nome do arquivo

O seu primeiro nome, em **letras minúsculas**, **sem acento**, terminando em `.json`:

```
ana.json   joao.json   maria-clara.json
```

Já existe alguém com o seu nome? Acrescente algo: `joao-2.json` ou `joao-p.json`.

Que formato é esse?

É **JSON**, um jeito de guardar dados que quase todo sistema entende. Apps do celular conversam com os servidores em JSON o tempo todo.

PRIVACIDADE

O mural e o repositório são **públicos**: qualquer pessoa na internet pode ver. E o Git guarda o **histórico**: até o que for apagado continua registrado. Por isso: **só o primeiro nome**. Nada de sobrenome, foto, telefone, e-mail, endereço ou rede social.

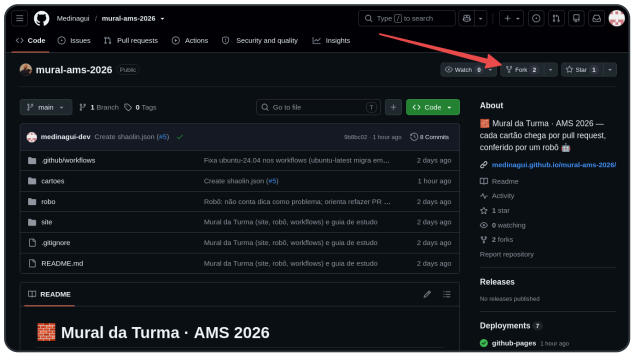
USE SEMPRE O BOTÃO FORK

Se você clicar em **Add file** direto no repositório do instrutor, o GitHub oferece um atalho para fazer o fork que **trava**. Siga o caminho desta apostila: primeiro o botão **Fork**, depois tudo na sua cópia.

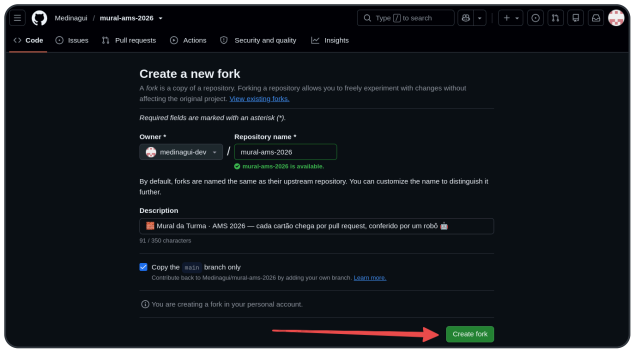
Passo a passo: o seu cartão

1 Faça a sua cópia do projeto (fork)

Entre em github.com/Medinagui/mural-ams-2026 e clique em **Fork**, no canto de cima à direita. Na tela seguinte, clique em **Create fork**.



02 O botão **Fork**.

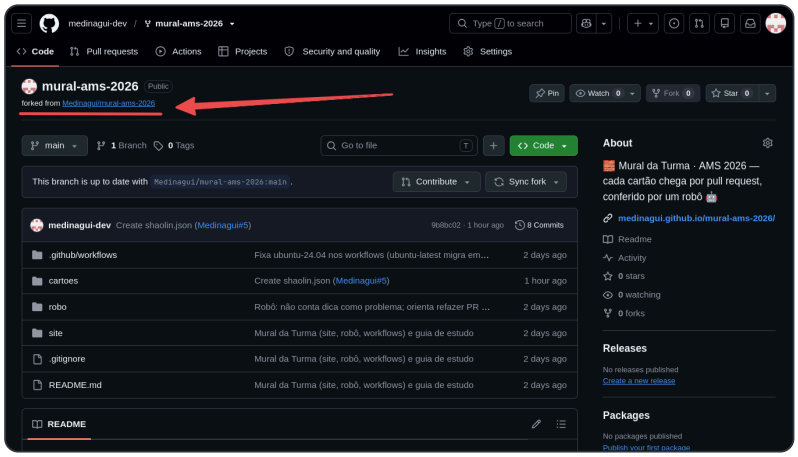


03 Não precisa mudar nada: **Create fork**.

2 Confira: você está na *sua* cópia

No topo aparece o **seu nome de usuário** antes de `mural-ams-2026` e, logo abaixo, *"forked from Medinagui/mural-ams-2026"*.

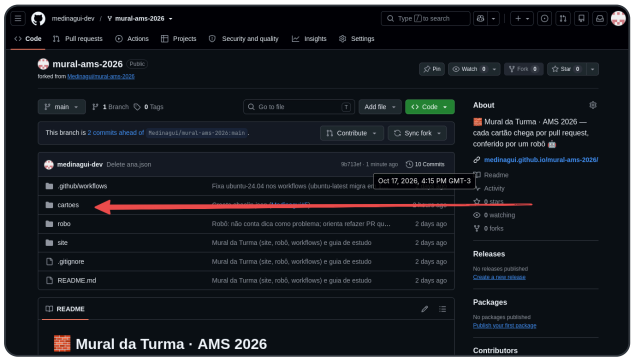
Daqui para frente, **todos os passos são aqui**, na sua cópia.



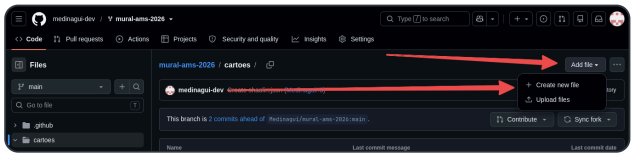
04 O fork. Aqui o usuário é `medinagui-dev`; no seu, aparece o seu.

3 Entre na pasta `cartoes` e crie um arquivo novo

Clique na pasta `cartoes`. Lá dentro, clique em **Add file** → **Create new file**.



14a A pasta `cartoes`.

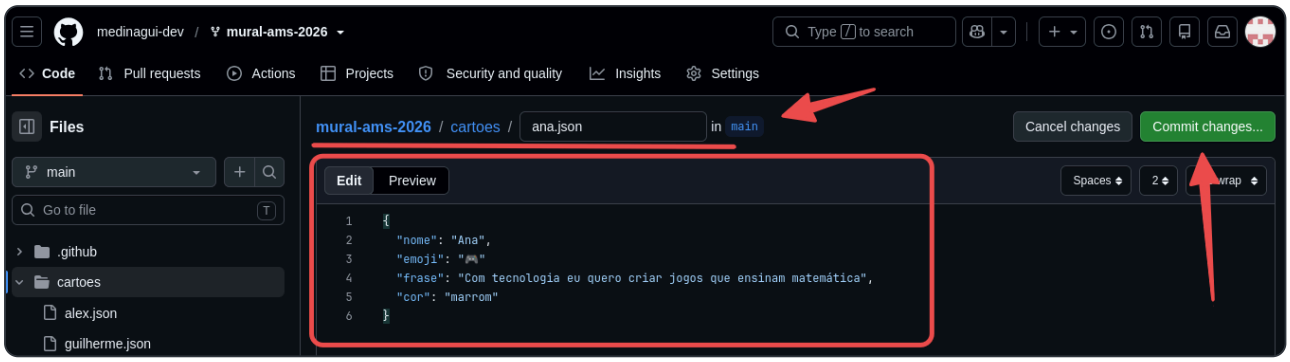


14b **Add file** → **Create new file**, já dentro de `cartoes`.

> **ATENÇÃO**
>
> O arquivo **tem** que ficar **dentro** da pasta `cartoes/`. Se ficar fora, o robô não aceita e você precisa recomeçar (página 7).

4 Dê nome ao arquivo e escreva o seu cartão

No campo do nome, escreva `seu-nome.json` (página 2). Confira se o caminho no topo é `mural-ams-2026 / cartoes / seu-nome.json`. Na área grande, cole o modelo e troque pelos seus dados.



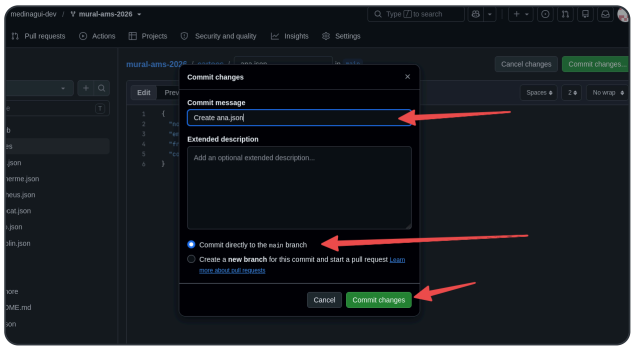
15 O editor. **Desafio:** este cartão tem **2 erros de propósito**, para mostrar o robô trabalhando. Você acha? (Resposta na página 6.)

5 Salve (commit)

Clique em **Commit changes...**. Na janela:

- escreva uma mensagem, como `Cartão da Ana`;
- deixe marcado **Commit directly to the main branch** (é a `main` da **sua cópia**, não a do instrutor);
- clique em **Commit changes**.

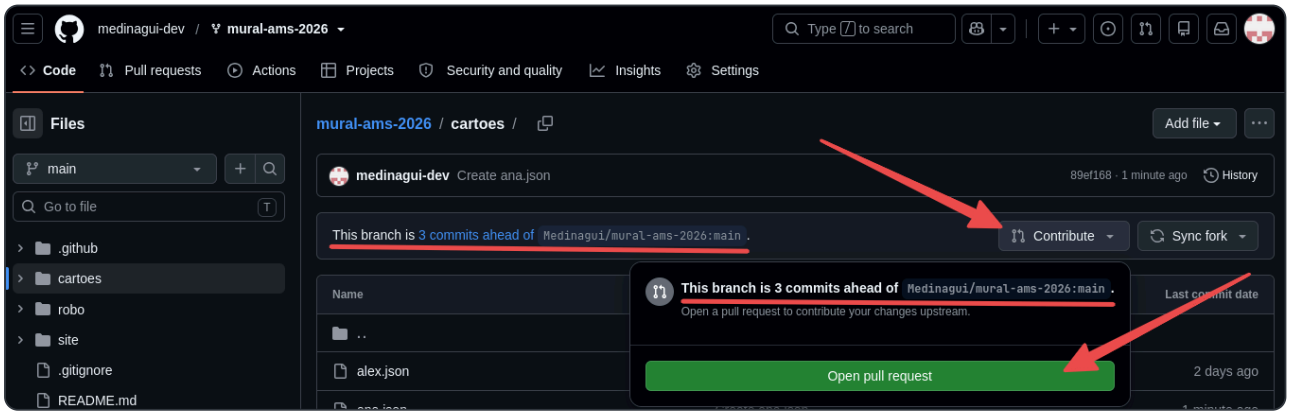
Pronto: você **salvou o jogo**.



16 A janela do commit.

6 Abra o pull request

Na sua cópia aparece a faixa **"This branch is 1 commit ahead of Medinagui/mural-ams-2026:main"**: a sua cópia tem 1 mudança a mais que o projeto original. Na mesma faixa, clique em **Contribute** → **Open pull request**.



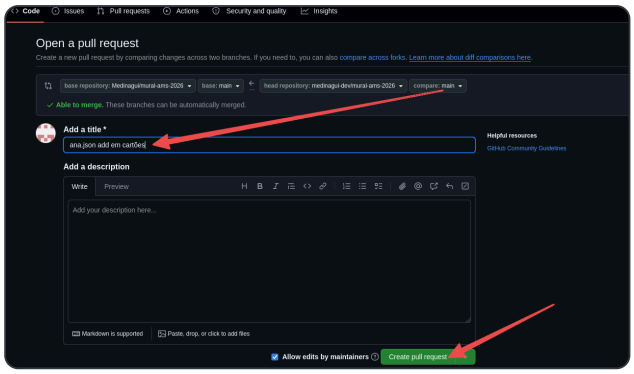
17 No print aparece "3 commits ahead" porque essa conta errou e apagou antes (página 7). Com você deve aparecer "1 commit ahead".

7 Create pull request

Na tela **Open a pull request**, confira o caminho no topo: da **sua cópia** (*head repository*) para a do instrutor (*base repository*).

Dê um título, como `Cartão da Ana`, e clique em **Create pull request**.

Pronto: você pediu para o instrutor **puxar** (*pull*) a sua mudança.



18 A tela do pull request.

8 Espere o robô

Em alguns segundos, o robô lê o seu cartão e comenta no pull request. Três respostas possíveis:

> ✅ **TUDO CERTO!**
>
> O robô mostra uma prévia do seu cartão. Agora é esperar o instrutor revisar e fazer o **merge**.

> 🔧 **QUASE LÁ!**
>
> O robô diz **o que** está errado e **como** consertar. Você conserta no próprio pull request (página 6).

> 🔁 **RECOMEÇAR**
>
> O arquivo ficou **fora** de `cartoes/` (ou você mexeu em outro arquivo). O robô explica como recomeçar (página 7).



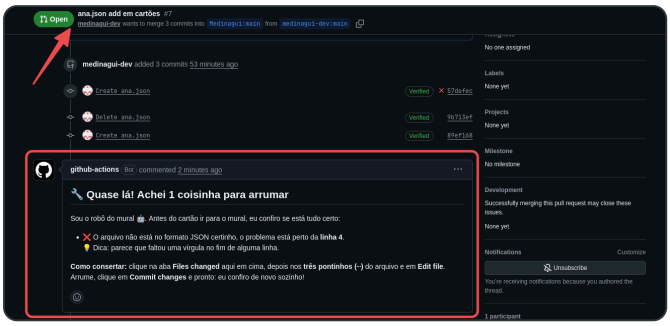
24 O robô aprovou.

9 Olhe para a TV

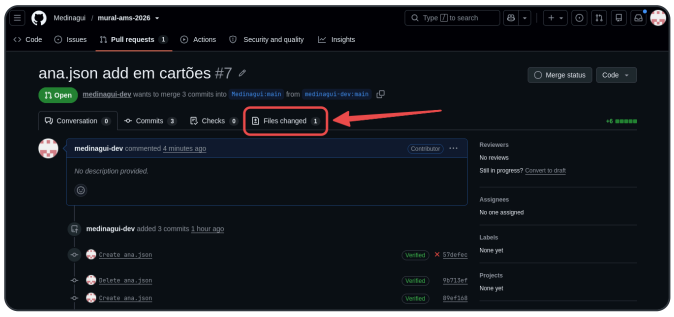
O instrutor lê cada frase (**revisão humana**) e faz o merge em lotes. Em mais ou menos **1 minuto**, o seu cartão aparece no mural!

O robô achou um erro: como consertar

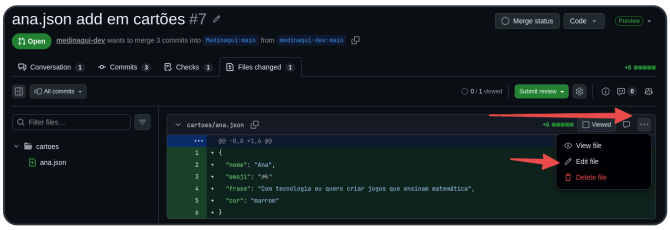
Não precisa fechar nada nem começar de novo. Você conserta **dentro do próprio pull request** e o robô confere de novo sozinho.



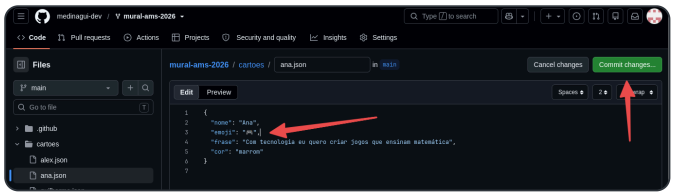
19 1º erro: *"parece que faltou uma vírgula no fim de alguma linha"*.



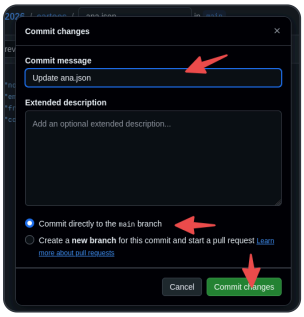
20a No alto do pull request, a aba **Files changed**.



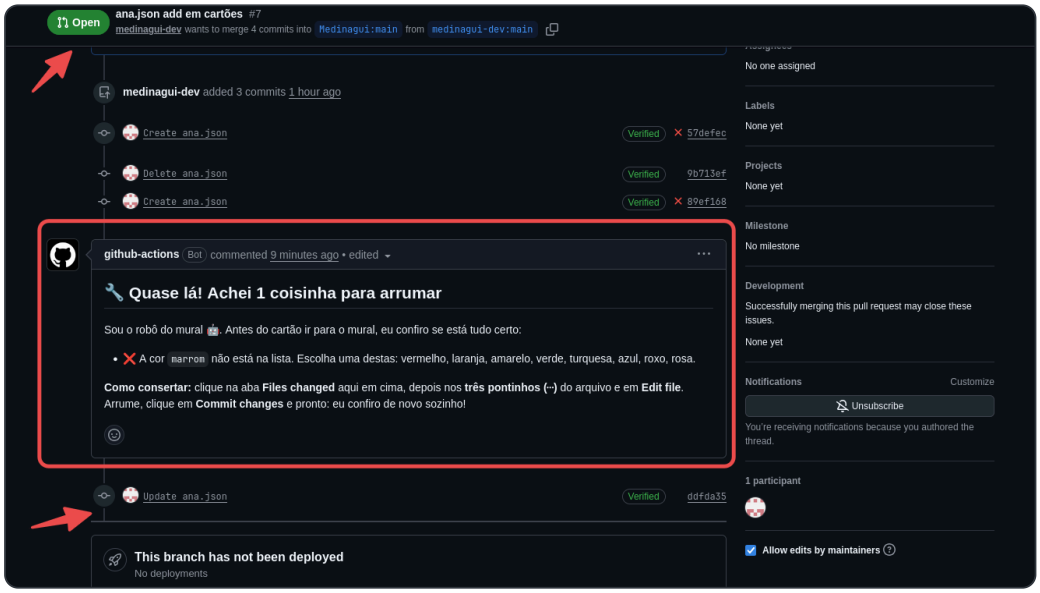
20b Nos **três pontinhos (···)** do arquivo → **Edit file**.



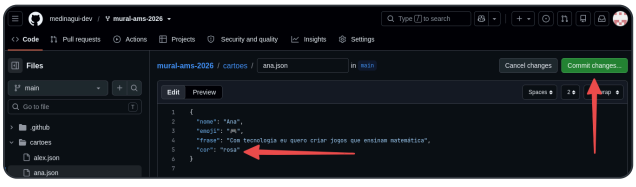
21a Arrume (aqui, a vírgula depois do emoji) e clique em **Commit changes...**



21b Confirme com **Commit changes**.



22 O robô conferiu de novo e achou o 2º erro: a cor `marrom` não existe.



23 Mesmo caminho: Edit file → `rosa` → Commit. Agora o robô dá ✅ (print 24, página 5).

ERROS MAIS COMUNS

- **Vírgula** faltando no fim de uma linha, ou sobrando na última antes do `}`.
- **Aspas curvas** `“ ”` (do celular ou do Word): use as retas `"`.
- **Campo com nome errado** (`emogi`): o robô sugere o certo.
- **Sobrenome**, frase com mais de 80 caracteres, cor fora da lista.

O arquivo ficou fora de `cartoes/`: recomeçar

Se você criou o arquivo no lugar errado, editar não resolve: precisa recomeçar. O robô responde assim:

github-actions Bot commented

🔧 Quase lá! Achei 1 coisinha para arrumar

Sou o robô do mural 🤖. Antes do cartão ir para o mural, eu confiro se está tudo certo:

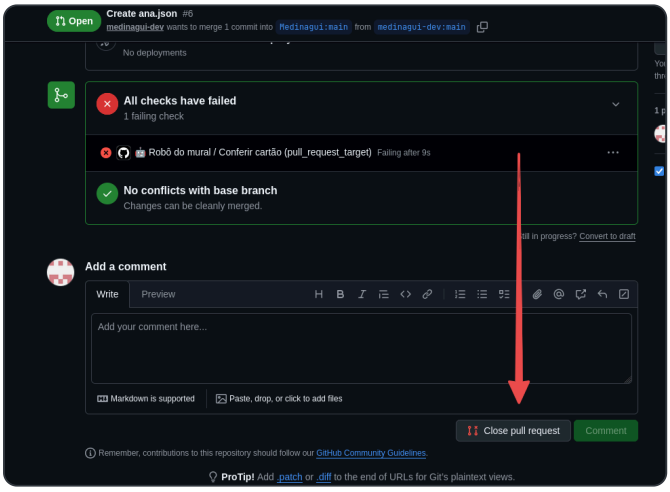
❌ Você mexeu em `ana.json`, mas o robô só aceita mudanças dentro da pasta `cartoes/`.

Como consertar: o jeito mais fácil é recomeçar.

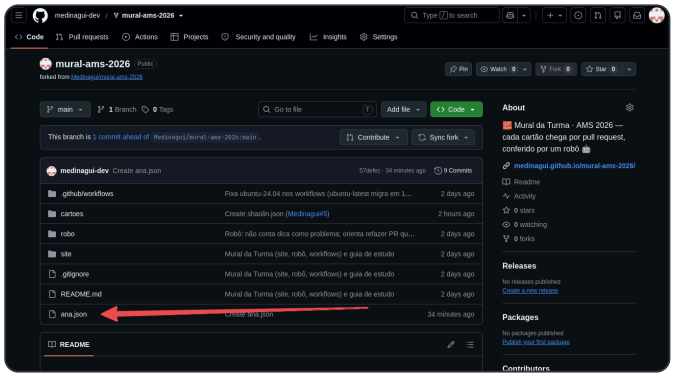
1. Lá embaixo, clique em **Close pull request**.
2. Vá para o **seu fork** (a sua cópia do projeto, que tem o seu nome de usuário no topo) e apague o arquivo que ficou errado: abra o arquivo → **três pontinhos (···)** → **Delete file** → **Commit changes**. Se você mexeu no cartão de outra pessoa, chame o instrutor.
3. Ainda no seu fork, entre na pasta `cartoes/` e crie **só** o seu arquivo (**Add file** → **Create new file**).
4. Clique em **Contribute** → **Open pull request** → **Create pull request**. Eu confiro de novo!

POR QUE PRECISA APAGAR?

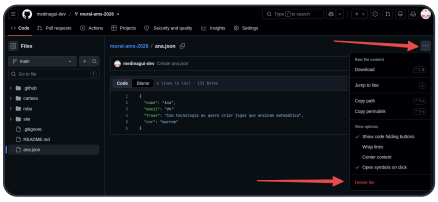
O arquivo errado continua na sua cópia. Se você não apagar, o pull request novo leva o erro junto e o robô reclama de novo.



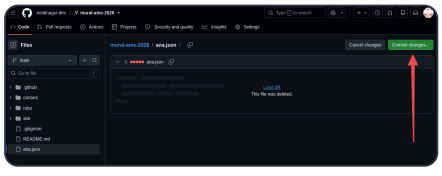
11 Passo 1: **Close pull request**, no fim da página do pull request.



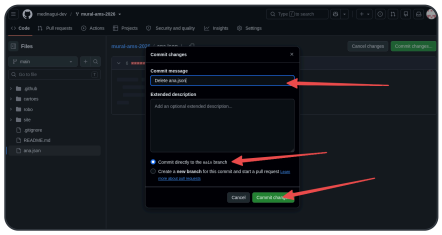
12a Passo 2: no seu fork, clique no arquivo que ficou fora de `cartoes`.



12b ··· → **Delete file**.



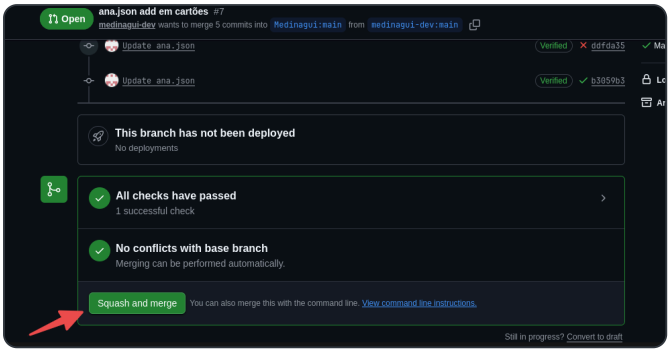
12c **Commit changes...**



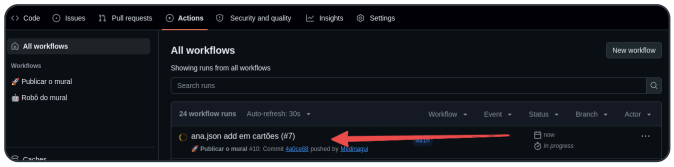
13 Confirme com **Commit changes**.

Depois, siga a partir do **passo 3** (página 3): pasta `cartoes` → Add file → Create new file → commit → Contribute → Open pull request.

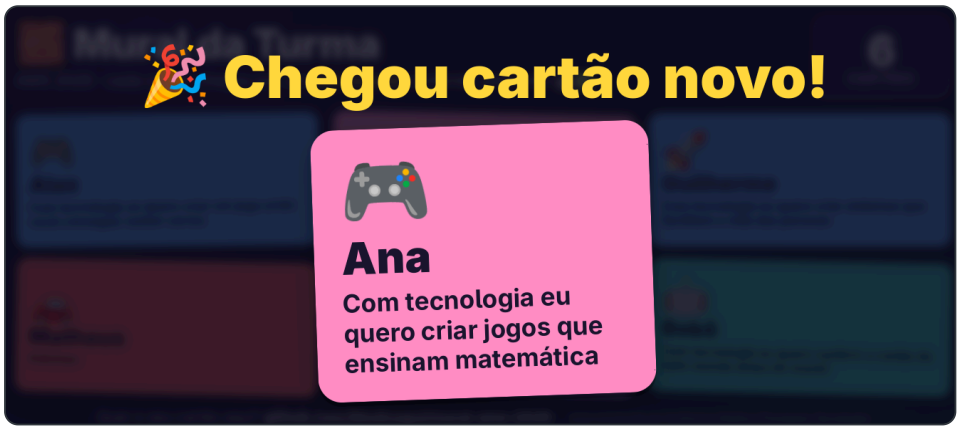
Depois do merge: o que aconteceu por trás



27 O instrutor revisa e clica em **Squash and merge**.



28 Na aba **Actions**, o robô "Publicar o mural" começa sozinho.



29 ~1 minuto depois, o cartão aparece no mural, em destaque.

CI · INTEGRAÇÃO CONTÍNUA

A **cada** pull request, um robô confere a mudança **antes** de juntar. Em empresas, ele roda centenas de testes. É o **fiscal da prova**: barra o que está fora das regras, mas quem decide é uma pessoa.

CD · ENTREGA CONTÍNUA

A **cada** merge, o sistema é **publicado sozinho**, sem ninguém copiar arquivo na mão. É a **esteira** do fast-food: o pedido aprovado vai sozinho até o balcão.

O robô **não é uma IA**: é um programa em JavaScript (Node.js) com regras fixas, rodando no **GitHub Actions**. Quer ver? Está na pasta `robo/` do repositório.

Front-end, back-end e banco de dados

NO RESTAURANTE	NO SISTEMA	NO MURAL
Cardápio e salão	**Front-end**: telas e botões (HTML, CSS, JavaScript)	A página do mural
Garçom	**API**: leva o pedido e traz a resposta	A API do GitHub, que o robô usa
Cozinha	**Back-end**: onde o trabalho acontece (Node, Express...)	O robô
Despensa	**Banco de dados**: onde os dados ficam guardados	A pasta `cartoes/`

O mural não é um sistema completo com front, back e banco: é uma imitação simples que montamos para funcionar dentro do GitHub.

E como saber se um sistema funciona? **Testes**: código que testa o próprio código. O robô do mural tem 17 testes que imitam erros de verdade (vírgula a mais, sobrenome, cor errada...).

Para fazer em casa: o seu próprio site

Com o **GitHub Pages**, você pode ter um site grátis no endereço `seu-usuario.github.io`. Tudo pelo navegador, em 10 minutos.

1 Crie um repositório com um nome especial

No GitHub, clique no **+** do canto de cima → **New repository**. Em *Repository name*, escreva **exatamente** `seu-usuario.github.io` (o seu nome de usuário + `.github.io`). Deixe **Public**, ative a opção de criar um **README** e clique em **Create repository**.

2 Crie a página

Add file → **Create new file**, com o nome `index.html`. Cole o modelo ao lado, troque pelo que é seu e clique em **Commit changes**.

A parte de cima (`<head>`) diz **como** a página aparece; a de baixo (`<body>`) diz **o que** aparece.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Site da Ana</title>
  <style>
    body {
      font-family: sans-serif;
      background: #ffe4f0;
      text-align: center;
    }
  </style>
</head>
<body>
  <h1>🎮 Oi, eu sou a Ana!</h1>
  <p>Quero criar jogos que
     ensinam matemática.</p>
</body>
</html>
```

3 Abra o seu site

Espere 1 ou 2 minutos e abra `seu-usuario.github.io`. Não apareceu? **Settings** → **Pages** → *Branch*: **main** → **Save**.

Para mudar: abra o `index.html`, clique no **lápis**, mude e faça o **commit**. O site se atualiza sozinho: é o **CD** de novo!

PRIVACIDADE

O seu site é **público**. Mesma regra do mural: nada de sobrenome, foto, telefone, endereço ou e-mail.

Continue aprendendo

- **Crie o seu próprio robô de CI/CD**: curso *Hello GitHub Actions* no **GitHub Skills** (skills.github.com), grátis e no navegador.
- **Git no seu computador**: Git (git-scm.com) ou **GitHub Desktop** (desktop.github.com), que faz commit e pull request com botões.
- **GitHub Student Developer Pack** (education.github.com): ferramentas grátis para estudantes. E inglês: programar treina!

Ideias de projeto

Portfólio
o seu site pessoal no GitHub Pages

Jogo no navegador
quiz ou jogo da memória em HTML e JavaScript

Mural do seu time
um fork deste mural para o time, a família ou os amigos

Site do grêmio ou do time
agenda, resultados, avisos

Bot de Discord
um robô que responde comandos no servidor da turma